# Übungsblatt 2

## Git, CSS und Evolution des Webs

5430UE Web and Data Engineering

22. April 2026

Prof. Dr. Michael Granitzer, Tobias Fuchs

# Lernziele

- Grundfunktion der Versionsverwaltung Git anwenden
- CSS Selektoren formulieren
- Evolution des Webs (1.0, 2.0, 3.0) verstehen

# Git

# Übung 2.1: FIM Account & FIM Gitlab

Für den Zugang auf das FIM Gitlab benötigen Sie einen FIM Account (siehe Blatt 01).

1. Stellen Sie sicher, dass Sie Zugang zum FIMGit haben. Melden Sie sich dafür mit Ihrer FIM-Kennung unter FIMGit an.
2. Machen Sie sich mit der Oberfläche vertraut. Sie werden später FIMGit nutzen, um eigene Repositories anzulegen.

# Übung 2.2: Einführung Git

Für die kommende Projektarbeit sollen Sie sich mit dem Versionsverwaltungs-Tool **Git** vertraut machen. Um ein Verständnis für die wichtigsten Befehle zu erhalten, arbeiten Sie das verlinkte Tutorial durch.

Wenn Sie sich keinen Bitbucket-Account anlegen wollen, können Sie auch direkt mit Ihrem FIMGit anstelle von BitBucket arbeiten. Erstellen Sie dazu das notwendige Repository in Ihrem FIMGit (unter *Projects*). Die folgenden Schritte lassen sich nun analog mit dem in FIMGit angelegten Repository anstelle eines Repositories auf BitBucket durchführen.

**Voraussetzung**: Falls Sie auf Ihrem System noch kein **Git** installiert haben, finden Sie unter https://git-scm.com/ den entsprechenden Download.

**Weitere interaktive Übungsmöglichkeit**: Bei weiterem Übungsbedarf können Sie das interaktive Tutorial absolvieren.

# Übung 2.2: Einführung Git — Lösung (1/5)

## Repository erstellen & klonen

- **Erstellen**: Anlegen des neuen Repositories über *Projects* > *New Project* > *Create blank project* im FimGit.
- **Klonen**: Navigieren Sie im Gitlab zu *Clone with HTTPS* und kopieren die URL.

```
git clone <kopierte URL>
```

- **Prüfen**: In Ihrem lokalen Order befindet sich nun der vom Server geklonte Repository-Ordner.

# Übung 2.2: Einführung Git — Lösung (2/5)

## Daten ändern & einfügen

- **Hinzufügen**: Legen Sie eine Datei `locations.txt` an.
- **Prüfen**: Zeigt ungespeicherte / neue Änderungen:

```
git status
```

- **Staging**: Fügt Datei zum nächsten Speicherpunkt (Commit) hinzu:

```
git add locations.txt
```

# Übung 2.2: Einführung Git — Lösung (3/5)

## Committen & Synchronisieren

- **Commit**: Speichert den Arbeitsstand mit einer Nachricht (`-m`):

```
git commit -m "Erster Commit von locations.txt"
```

- **Push**: Überträgt Ihre lokalen Commits zum FimGit-Server:

```
git push origin main
```

- **Pull**: Neue Änderungen von anderen (oder über die Gitlab UI) ins lokale Repository laden:

```
git pull
```

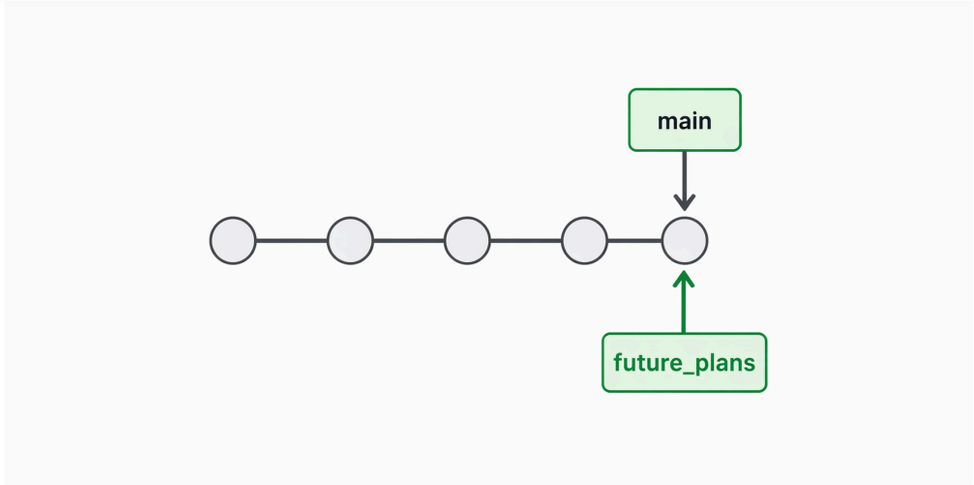
# Übung 2.2: Einführung Git — Lösung (4/5)

## Branches in Git

Ein **Branch** ist ein paralleler Entwicklungszweig zur isolierten Entwicklung von Features, ohne den Hauptzweig (`main`) zu stören.

- **Branch anlegen** (`git branch`): Erzeugt einen neuen Branch.

```
git branch future_plans
```

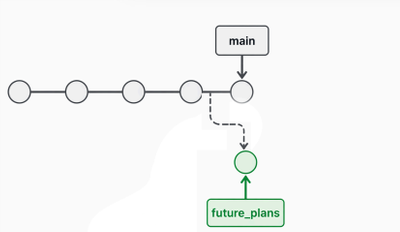


# Übung 2.2: Einführung Git — Lösung (5/5)

## Auf Branches arbeiten & mergen

- **Auschecken**: Auf den Branch wechseln. *(Modern: `git switch`)*

```
git checkout future_plans
```




- **Mergen**: Führt Änderungen in den Hauptzweig zurück. Zuerst auschecken (`git checkout main`), dann mergen:

```
git merge future_plans
```

- Danach den alten Branch oft löschen (`git branch -d future_plans`) und pushen.

# Übung 2.3: Verwendung von FIMGit

1. Erstellen Sie in FIMGit ein neues Repository `wde-exercise01`.
2. Klonen Sie das Repository lokal.
3. Erstellen Sie die Datei `hello.txt` mit dem Inhalt *Hallo WDE!*, commiten Sie die Datei und pushen anschließend den Commit. Notieren Sie alle für diese Teilaufgabe notwendigen Befehle!

# Übung 2.3: Verwendung von FIMGit — Lösung

Teilaufgabe 3:

```
echo "Hallo WDE!" > hello.txt
git add hello.txt
git commit -m "Add hello.txt"
git push origin main
```

**Warum diese Reihenfolge?**

- `git add` überführt die Datei in den Staging-Bereich
- `git commit` erzeugt den Snapshot
- `git push` überträgt den Commit an den Remote-Server

# Übung 2.4: Git Grundlagen - Branching

Angenommen Sie arbeiten an einem Feature `user-login` in einem Projekt.

1. Geben Sie die Git-Befehle an, die nötig sind, um einen neuen Branch `feature/user-login` zu erstellen und direkt zu wechseln?
2. Erklären Sie kurz den Unterschied zwischen `git merge` und `git rebase`?
3. Beim GitFlow-Prinzip wird empfohlen, Arbeiten in Feature-Branches auszulagern statt direkt auf dem `main`-Branch zu arbeiten. Erklären Sie kurz, warum das sinnvoll ist.

# Übung 2.4: Branching — Lösung (1/2)

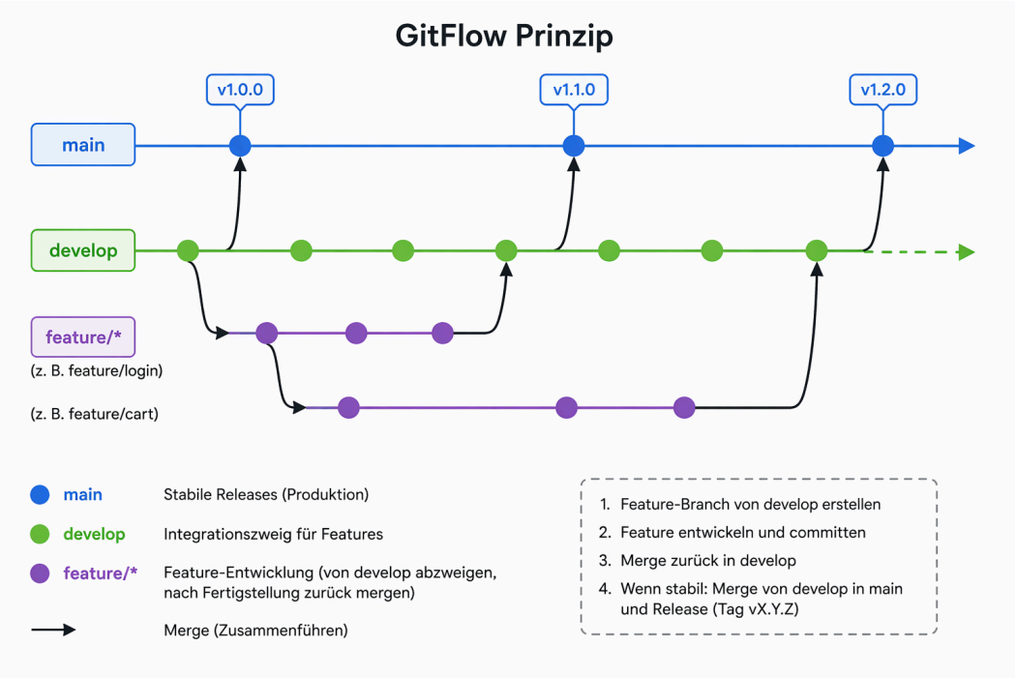
1. Erstellung des Branches:

```
git checkout -b feature/user-login
```

2.
- `git merge`: Kombination aus dem Quell-Branch in den Ziel-Branch. Git erstellt einen speziellen Merge-Commit, der zwei Eltern-Historien miteinander verknüpft.
- `git rebase`: Nimmt die Commits des Feature-Branches und setzt sie "oben auf" die aktuelle Spitze des Ziel-Branches. Dabei wird die **Historie des Feature-Branches verändert**, da dessen Commits neu geschrieben werden, um eine lineare Abfolge ohne Merge-Commit zu erzeugen. Es ist so, als hätte die Arbeit gerade erst auf Basis der neuesten Änderungen begonnen.

# Übung 2.4: Branching — Lösung (2/2)

3. Feature-Branches isolieren Arbeiten voneinander:
   - `main` bleibt jederzeit deploybar
   - Parallele Features stören sich nicht
   - Code-Reviews über Merge Requests sind vor dem Merge möglich



# Evolution des Webs

# Übung 2.5: Evolution des Webs — 1.0, 2.0, 3.0

Das Web hat sich in Generationen entwickelt. Durchforsten Sie das Internet nach Informationen zu den Bereichen **Web 1.0**, **Web 2.0** sowie **Web 3.0**.

Füllen Sie die Tabelle aus und erklären Sie welches Web-Paradigma heute in Unternehmensanwendungen dominiert.

| Merkmal | Web 1.0 | Web 2.0 | Web 3.0 |
|---|---|---|---|
| **Inhaltsproduktion** | | | |
| **Nutzerinteraktion** | | | |
| **Typische Technologien** | | | |
| **Beispiel-Anwendung** | | | |

# Übung 2.5: Evolution des Webs — Lösung (1/3)

| Merkmal | Web 1.0 | Web 2.0 |
|---|---|---|
| **Inhaltsproduktion** | Statisch und zentral; Inhalte werden fast nur von Betreibern erstellt ("Read-Only Web"). | User-Generated Content; Nutzer werden vom Konsumenten zum Produzenten ("Prosumer"). |
| **Nutzerinteraktion** | Passiver Konsum (Lesen) | Fokus auf Interaktion (Liken, Teilen, Kommentieren) und aktive Kollaboration in Echtzeit. |
| **Typische Technologien** | Statisches HTML, FTP, einfache Grafik-Formate. | **AJAX** (Asynchrones Laden von Daten ohne Seiten-Neuladen), **JSON** (Leichtgewichtiges Format zum Datenaustausch). |
| **Beispiel-Anwendung** | Persönliche Homepages, Online-Branchenbücher. | Wikipedia, Instagram, Blogs. |

# Übung 2.5: Evolution des Webs — Lösung (2/3)

| Merkmal | Web 3.0 |
| --- | --- |
| **Inhaltsproduktion** | Dezentral und KI-gestützt; Inhalte werden oft durch KI-Modelle generiert oder semantisch veredelt. |
| **Nutzerinteraktion** | Intelligent und autonom; Maschinen verstehen den Kontext der Daten, den Sinn der Interaktion und passen sich automatisch an. Interaktionen erfolgen oft über dezentrale Identitäten statt über zentralisierte Konten, bei denen Nutzer ihre Identität und Daten zur Verwaltung an große Plattform-Betreiber (wie Google oder Meta) abtreten müssen. |
| **Typische Technologien** | **RDF** (Framework zur Beschreibung von Datenbeziehungen), **Blockchain** (Dezentrale, manipulationssichere Datenspeicherung). |
| **Beispiel-Anwendung** | Semantische Suche (WolframAlpha), KI-Agenten. |

# Übung 2.5: Evolution des Webs — Lösung (3/3)

**Dominierendes Web-Paradigma:**

Heute dominiert das Web-2.0-Paradigma (Social Web) in Unternehmensanwendungen: Interaktive Plattformen, Cloud-Kollaboration und Single-Page-Applications. Web-3.0-Ansätze (wie semantische Datenaufbereitung oder KI-Integrationen) werden jedoch zunehmend zur Differenzierung genutzt.

*Web 3.0*: Web 3.0 bezeichnet die nächste Generation des Internets, die auf Dezentralisierung (z.B. durch Blockchain), mehr Nutzerkontrolle über Daten und intelligente Systeme setzt. Es soll ein offeneres, transparenteres und stärker personalisiertes Web ermöglichen, oft unterstützt durch Technologien wie KI.

# CSS

# Übung 2.6: Wiederholung CSS

Aus der Veranstaltung *Einführung in Internet Computing* kennen Sie bereits die Grundlagen von CSS.

Bearbeiten Sie die 32 Level aus dem CSS Dinner https://flukeout.github.io/, um Ihre Kenntnisse aufzufrischen und die Grundlagen für CSS Selektoren zu schärfen.

# Übung 2.6: Wiederholung CSS — Lösung (1/5)

| Konzept | Erklärung | Beispiel |
| --- | --- | --- |
| **Basis-Selektoren** | Typ, Klasse, ID oder alle Elemente. | `div`, `.class`, `#id`, `*` |
| **Kombinatoren** | Beziehungen zwischen Elementen (Nachfahren, direkte Kinder, benachbarte Geschwister, allgemeine Geschwister). | `A B`, `A > B`, `A + B`, `A ~ B` |
| **Pseudo-Klassen** | Auswahl basierend auf Position, Typ oder Zustand. | `:first-child`, `:nth-of-type(n)`, `:not()` |
| **Attribut-Selektoren** | Auswahl basierend auf Attributen. | `[attr]`, `[attr="val"]`, `[attr^="val"]` |
| **Gruppierung** | Mehrere Selektoren gleichzeitig ansprechen. | `A, B` |

# Übung 2.6: Wiederholung CSS — Lösung (2/5)

1. Auswahl nach Typ des Elements (Hier: `plate`):
   ```
   plate
   ```
2. Auswahl nach Typ des Elements (Hier: `bento`):
   ```
   bento
   ```
3. Auswahl eines Elements mit ID (Hier: Element mit ID `fancy`):
   ```
   #fancy
   ```
4. Nachfolger-Elemente auswählen (Hier: Wählt alle `apple`, die Nachfahren (also innerhalb) eines `plate` sind.):
   ```
   plate apple
   ```
5. Kombination von Nachfolger und ID (Hier: Alle `pickle` innerhalb des Elements mit ID `#fancy`):
   ```
   #fancy pickle
   ```
6. Auswahl aller Elemente mit Klasse (Hier: Klasse `small`):
   ```
   .small
   ```
7. Kombination von Klasse und Typ (Hier: Alle `orange` mit Klasse `small`):
   ```
   orange.small
   ```
8. Kombination bisheriger Inhalte: Kleine `orange` in `bento`-Boxen:
   ```
   bento orange.small
   ```

# Übung 2.6: Wiederholung CSS — Lösung (3/5)

9. Liefert alle Elemente der aufgeführten Typen (Hier: `plate` und `bento`):
   `plate, bento`
10. Wähle alle Elemente: `*`
11. Wähle alle Elemente die Nachfolger von `plate` sind:
    `plate *`
12. Selektor für direkt benachbarte Geschwister(gleiche Hierarchieebene und direkt aufeinanderfolgend) (Hier: `apple` direkt nach `plate`):
    `plate + apple`
13. Allgemeiner Geschwister-Selektor (gleiche Hierarchieebene, irgendwo danach) (Hier: Alle `pickle` nach einem `bento`):
    `bento ~ pickle`
14. Wähle direkte Kinder, d.h. direkt innerhalb (Hier: `apple` auf `plate`):
    `plate > apple`
15. Erstes Kindelement (Hier: `orange`, die das erste Kind sind):
    `orange:first-child`
16. Elemente, die einziges Kind sind (Hier: `plate` als einziges Kind):
    `plate:only-child`

# Übung 2.6: Wiederholung CSS — Lösung (4/5)

17. Letztes Element innerhalb eines anderen (Hier: kleine Elemente als letztes Kind):
    `.small:last-child`
18. Wähle jedes dritte Element innerhalb eines anderen Elements:
    `:nth-child(3)`
19. Auswahl von hinten gezählt (Hier: `bento`, die drittletztes Kind sind):
    `bento:nth-last-child(3)`
20. Wählt erstes Element eines Typs (Hier: Erster `apple`):
    `apple:first-of-type`
21. Wähle gerade `plate`:
    `plate:nth-of-type(even)`
22. Formelbasierte Auswahl (`2n+3`): Jedes zweite `plate`, startend beim dritten:
    `plate:nth-of-type(2n+3)`
23. Einziges Element seines Typs (Hier: `apple` als einziges Element in `plate`):
    `plate apple:only-of-type`
24. Letztes Element eines Typs (Hier: jeweils letzter `apple` und `orange`):
    `apple:last-of-type, orange:last-of-type`

# Übung 2.6: Wiederholung CSS — Lösung (5/5)

25. Wählt Elemente ohne Kindelemente (Hier: leere `bento`):
    `bento:empty`
26. Negationsselektor (Hier: `apple` ohne die Klasse `small`):
    `apple:not(.small)`
27. Wähle alle Elemente mit Attribut `for`:
    `*[for]`
28. Wähle alle `plate` mit `for`-Attribut:
    `plate[for]`
29. Attributwert-Selektor (Hier: `bento` mit `for="Vitaly"`):
    `bento[for="Vitaly"]`
30. Attribut beginnt mit (Hier: `for`-Attribut beginnt mit `"Sa"`):
    `*[for^="Sa"]`
31. Attribut endet mit (Hier: `for`-Attribut endet mit `"ato"`):
    `*[for$="ato"]`
32. Attribut enthält (Hier: `for`-Attribut enthält `"obb"`):
    `*[for*="obb"]`

# Übung 2.7: CSS - Anwendung (1/2)

Betrachten Sie folgenden HTML-Code:

```
<!DOCTYPE html>
<html>
<head>
  <title>TODO Liste</title>
</head>
<body>
  <h1>Aufgaben:</h1>
  <p>Folgende Aufgaben muessen <strong>unbedingt</strong> erledigt werden.</p>
  <ul>
    <li class="styles" for="basic" id="entry01">Aufraeumen<input type="checkbox" checked></li>
    <li>Einkaufen</li>
    <li for="basicStudies">Lernen
      <ol>
        <li id="entry02">HTML<input type="checkbox" checked></li>
        <li>CSS<input type="checkbox"></li>
      </ol>
    </li>
    <li for="peace">Entspannen<input type="checkbox" checked></li>
```

# Übung 2.7: CSS - Anwendung (2/2)

Erweitern Sie den <head> um ein Style-Element, mit denen Sie folgende Darstellungsänderungen durchführen:

1. Erhöhen Sie die Schriftgröße für alle Listeneinträge ohne Klasse "styles" auf 20 Pixel.
2. Blenden Sie das zweite Element der TODO-Liste und der Unterliste "Lernen" aus.
3. Färben Sie "unbedingt" in Rot.
4. Drehen Sie den "Aufgaben:" Header um 10 Grad an der Z Achse. Verschieben Sie den Header um 40 Pixel nach rechts und 150 Pixel nach unten.
5. Streichen Sie den Text mit Attributen "basic" und "basicStudies" mit Hilfe eines einzelnen Selektors durch.

# Übung 2.7: CSS - Anwendung — Lösung

```css
li:not(.styles) {
  font-size: 20px;
}

li:nth-child(2) {
  display: none;
}

strong {
  color: red;
}

h1 {
  transform: rotateZ(10deg) translate(40px, 150px);
}

li[for^="basic"] {
  text-decoration: line-through;
}
```



# Materialien

# Materialien zum Übungsblatt

- todo_list_template.html